

NexusCRM Developer Guide

NexusCRM

Enterprise SaaS CRM Developer Guide

Version 1.0

Prepared for product engineers, platform developers, QA teams, and deployment operators.

Document Identity

Product: NexusCRM

Repository: NexusCRM

Application Type: Multi-tenant enterprise SaaS CRM

Frontend: Next.js 16 App Router, React 19, Tailwind CSS 4

Backend: NestJS 11 JavaScript API, Mongoose, Passport JWT, CASL RBAC

Data Platform: MongoDB with tenant-scoped collections

Realtime Layer: Socket.IO chat and notifications

Deployment Model: Vercel frontend, Render backend, MongoDB Atlas database

Table Of Contents

- Chapter 1: Product Vision And Problem Statement
- Chapter 2: System Architecture
- Chapter 3: Repository And Directory Structure
- Chapter 4: Local Development Setup
- Chapter 5: Environment Configuration
- Chapter 6: Tenancy, Authentication, Sessions, And RBAC
- Chapter 7: Backend Engineering Guide
- Chapter 8: Frontend Engineering Guide
- Chapter 9: Module-By-Module Functional Guide
- Chapter 10: API Route Reference
- Chapter 11: Data Model And Persistence
- Chapter 12: Realtime, Chat, Files, And Notifications
- Chapter 13: Billing, Plans, Trials, And SaaS Operations
- Chapter 14: Integrations And External Services
- Chapter 15: Deployment Guide
- Chapter 16: Validation, QA, And Smoke Testing
- Chapter 17: Troubleshooting And Operational Playbooks
- Chapter 18: Contribution Standards And Extension Patterns

- [Appendix A: Quick Command Reference](#)
- [Appendix B: Developer Checklists](#)

Chapter 1

Product Vision And Problem Statement

Chapter 1: Product Vision And Problem Statement

Problem Statement

Revenue teams often operate across disconnected tools: one application for leads, another for deals, a separate ticketing system for service, spreadsheets for reporting, external email inboxes for communication, and manual workflows for billing or operations. This creates operational friction:

- Customer context is fragmented across sales, marketing, service, and finance.
- Managers cannot see a single source of truth for pipeline, work, support, and billing.
- Tenant-level permissions and SaaS billing are hard to maintain when CRM features grow.
- Import/export workflows, audit history, notifications, and automation are frequently treated as afterthoughts.
- Scaling from a small workspace to an enterprise workspace requires stronger RBAC, settings, operational modules, and deployment discipline.

NexusCRM Solution

NexusCRM is designed as an enterprise-grade, multi-tenant SaaS CRM that combines workspace management, CRM records, sales operations, service desk workflows, marketing communication, analytics, billing, automation, activity tracking, security controls, file management, chat, and realtime notifications into one platform.

The solution is built around four principles:

- **Tenant isolation by default:** every customer workspace is separated through `tenantId`, tenant-aware routing, and guard-level access checks.
- **Operational completeness:** CRM records are not just CRUD pages; they include workflows, activity, line items, conversions, imports, exports, documents, and notifications.
- **Enterprise governance:** RBAC, audit events, security center controls, data jobs, and settings modules are first-class parts of the system.
- **Deployable SaaS architecture:** the application is structured for Vercel frontend deployment, Render backend deployment, MongoDB Atlas persistence, and optional integration providers.

Functional Areas

NexusCRM currently includes:

- Tenant workspaces and onboarding.
- Login, signup, session restore, tenant switching, and superadmin access.
- Role, group, department, and permission management.
- CRM objects: leads, contacts, companies, deals, and requests.
- Lead dedupe, routing, conversion, and customer 360/account workspace.
- Sales pipelines, opportunity line items, products, quotations, orders, invoices, conversions, and PDF generation.
- Service desk tickets, queues, macros, replies, notes, routing, SLA state, and activity recording.
- Shared inbox, email accounts, mass mail, SMTP/IMAP, Gmail OAuth, and unsubscribe handling.
- Projects, tasks, memos, comments, subtasks, review, and conversion flows.
- Automation rules, runs, actions, conditions, retries, and execution history.
- Reports, analytics, export jobs, data import/export jobs, files, security center, audit stream, and notifications.
- Realtime chat, attachments, emoji support, read receipts, and notification bell.

Developer Responsibility

Developers working on NexusCRM should treat every feature as part of a tenant-scoped SaaS system. That means:

- Never query tenant-owned data without a tenant boundary.
- Prefer existing services, schemas, API helpers, list pages, and settings primitives.
- Keep frontend permission gates aligned with backend RBAC.
- Add activity or notification events for important user-facing actions.
- Validate deployment behavior for both local development and hosted environments.

Footer: NexusCRM Developer Guide | Chapter 1 | Product Vision And Problem Statement

Chapter 2

System Architecture

Chapter 2: System Architecture

Architecture Overview

NexusCRM uses a separated frontend/backend architecture:

- The **frontend** is a Next.js 16 App Router application in frontend/.
- The **backend** is a NestJS 11 JavaScript API in backend/.
- MongoDB stores application data in a single database with tenant-scoped documents.
- Socket.IO powers realtime object chat and in-app notifications.
- Vercel hosts the frontend and proxies browser /api/* calls to the backend.
- Render hosts the backend and exposes /api/* plus /socket.io.

High-Level Request Flow

```
Browser
-> Vercel Next.js app
-> frontend/app/api/[...path]/route.js proxy
-> Render NestJS API
-> TenantGuard + TenantScopeInterceptor
-> Controller
-> Service
-> Mongoose Model
-> MongoDB Atlas
```

Runtime Responsibilities

Layer	Responsibility
Next.js App Router	Pages, layouts, route proxy, static marketing, tenant workspace UI
React Query	Client-side data fetching, list refreshes, mutations
Local Storage Session	JWT, active tenant subdomain, tenant id, CASL rules
NestJS Controllers	HTTP route boundaries and request binding
NestJS Services	Business logic, workflows, validation, aggregation
Mongoose Schemas	Data shape, indexes, tenant-scoped persistence
Guards/Policies	Authentication, tenant membership, roles, and CASL permissions
Socket.IO	Object rooms, user rooms, chat, typing, notifications, read receipts
File Storage	Local file artifacts with metadata in MongoDB
Stripe/Brevo/Gmail/SMTP	Optional external provider integrations

Backend Entry Points

The backend starts in backend/src/main.js.

Main startup behavior:

- Loads reflect-metadata.

- Configures DNS resolver fallback for MongoDB Atlas SRV resolution.
- Creates the Nest application.
- Applies a larger JSON body limit through `REQUEST_BODY_LIMIT`.
- Preserves raw body parsing for `/api/billing/webhook`.
- Sets global API prefix `/api`.
- Initializes Socket.IO using the JWT secret.
- Enables CORS for local and configured frontend origins.
- Ensures the default superadmin account exists.
- Exposes `/api/health`.
- Listens on `PORT` or `4000`.

The root module is `backend/src/app.module.js`.

Root module responsibilities:

- Imports all business modules.
- Connects Mongoose using `MONGODB_URI`.
- Registers global tenant guard, tenant middleware, and tenant scope interceptor.
- Wires cross-cutting modules such as auth, RBAC, CRM, mail, realtime, files, data jobs, security, subscription, and extensions.

Frontend Entry Points

The frontend starts from `frontend/app/layout.js`.

Main frontend layers:

- `ThemeProvider` handles theme preferences.
- `NotificationProvider` renders toast notifications.
- Tenant pages use `frontend/app/[tenant]/layout.js`.
- Tenant pages are gated by `TenantGate`.
- Workspace pages render through `AppShell`, `AppSidebar`, and `AppTopbar`.
- API calls use `frontend/lib/api.js`.
- Browser API calls go through the same-origin proxy at `frontend/app/api/[...path]/route.js`.

Architecture Rules

- Public pages may call public APIs directly.
- Tenant pages must be inside `TenantGate`.
- Frontend navigation items should include action/subject metadata for RBAC visibility.
- Backend controllers that expose sensitive data must use guards and policy handlers.
- Tenant-owned models should include `tenantId` and useful compound indexes.
- Long-running operations should use `DataJob` or dedicated operational services.

Footer: NexusCRM Developer Guide | Chapter 2 | System Architecture

Chapter 3

Repository And Directory Structure

Chapter 3: Repository And Directory Structure

Root Structure

```
CRM/
├── backend/
│   ├── package.json
│   ├── scripts/
│   └── src/
├── frontend/
│   ├── app/
│   ├── components/
│   ├── lib/
│   ├── package.json
│   └── public/
├── docs/
│   ├── CRM_MODULE_GAP_AUDIT.md
│   ├── DEPLOY.md
│   ├── ENTERPRISE_CRM_EXPANSION_REPORT.md
│   ├── INTEGRATIONS-TESTING.md
│   └── NEXUSCRM_DEVELOPER_GUIDE.md
├── .env.example
├── package.json
├── render.yaml
└── README.md
```

Root Files

Path	Purpose
README.md	Product overview, stack, quick start, tenancy model
.env.example	Shared reference for frontend and backend environment variables
package.json	Root helper scripts for installing and running both apps
render.yaml	Render deployment blueprint for the backend
docs/	Product reports, deployment notes, integration testing, developer guide

Root Scripts

```
{
  "dev:frontend": "npm run dev --prefix frontend",
  "dev:backend": "npm run start:dev --prefix backend",
  "install:all": "npm install --prefix frontend && npm install --prefix backend"
}
```

Backend Structure

```
backend/src/
├── app.module.js
├── main.js
├── common/
│   ├── casl/
│   ├── constants/
│   ├── decorators/
│   ├── guards/
│   ├── interceptors/
│   ├── middleware/
│   ├── policies/
│   ├── providers/
│   └── utils/
└── modules/
```

```

■   ■■■ activity/
■   ■■■ analytics/
■   ■■■ auth/
■   ■■■ crm/
■   ■■■ dashboard/
■   ■■■ data-jobs/
■   ■■■ email/
■   ■■■ extensions/
■   ■■■ files/
■   ■■■ jobs/
■   ■■■ mail/
■   ■■■ metadata/
■   ■■■ public/
■   ■■■ rbac/
■   ■■■ realtime/
■   ■■■ security/
■   ■■■ subscription/
■   ■■■ superadmin/
■   ■■■ tasks/
■   ■■■ tenant/
■   ■■■ voip/
■■■ realtime/
    ■■■ socket-hub.js

```

Frontend Structure

```

frontend/
■■■ app/
■   ■■■ (auth)/
■   ■■■ [tenant]/
■   ■■■ api/
■   ■■■ superadmin/
■   ■■■ globals.css
■   ■■■ icon.svg
■   ■■■ layout.js
■■■ components/
■   ■■■ chat/
■   ■■■ crm/
■   ■■■ dashboard/
■   ■■■ layout/
■   ■■■ marketing/
■   ■■■ notifications/
■   ■■■ providers/
■   ■■■ sales/
■   ■■■ service/
■   ■■■ settings/
■   ■■■ tasks/
■   ■■■ ui/
■■■ lib/
■   ■■■ api.js
■   ■■■ ability.js
■   ■■■ navigation.js
■   ■■■ socket.js
■   ■■■ *-api.js
■■■ scripts/

```

Directory Design Principles

- Backend modules should keep schemas, controllers, services, and module registration together.
- Frontend feature pages should use shared API helpers instead of direct fetch logic.
- Shared UI components should live in `components/` rather than inside route folders when reused.
- Generic CRM and extension pages should reuse `CrmListPage`, `ModuleListPage`, and module config files.
- Documentation belongs in `docs/` and should reference code paths exactly.

Footer: NexusCRM Developer Guide | Chapter 3 | Repository And Directory Structure

Chapter 4

Local Development Setup

Chapter 4: Local Development Setup

Prerequisites

- Node.js 20 or newer.
- npm.
- MongoDB Atlas URI or a local MongoDB instance.
- GitHub repository access.
- Optional provider accounts for Brevo, Stripe, Google OAuth, reCAPTCHA, SMTP/IMAP, and Zadarma.

Install Dependencies

From the repository root:

```
npm run install:all
```

This installs packages for both:

- frontend/
- backend/

Create Environment Files

```
cp .env.example backend/.env
cp .env.example frontend/.env.local
```

On Windows PowerShell:

```
Copy-Item .env.example backend/.env
Copy-Item .env.example frontend/.env.local
```

Run Backend

```
npm run dev:backend
```

Equivalent:

```
cd backend
npm run start:dev
```

Default backend URL:

```
http://localhost:4000
```

Health endpoint:

```
http://localhost:4000/api/health
```

Run Frontend

```
npm run dev:frontend
```

Equivalent:

```
cd frontend
npm run dev
```

Default frontend URL:

```
http://localhost:3000
```

Local Tenant Access

The app supports tenant path routing such as:

```
http://localhost:3000/{tenant}/dashboard
```

Subdomain-style routing is also supported by the tenant model and middleware when `APP_DOMAIN` / `NEXT_PUBLIC_APP_DOMAIN` are configured for a domain that supports subdomains.

First User Flow

- Start backend.
- Start frontend.
- Visit `/signup` or `/register`.
- Create a workspace.
- Verify email if verification provider is configured.
- Complete onboarding.
- Use `/[tenant]/dashboard`.

Superadmin Flow

The backend bootstraps a superadmin using:

- `SUPERADMIN_EMAIL`
- `SUPERADMIN_PASSWORD`
- `SUPERADMIN_TENANT_SUBDOMAIN`

Superadmin access is available at:

```
/superadmin
```

Local Validation Commands

```
npm run lint --prefix frontend
npm run build --prefix frontend
npm run start --prefix backend
```

Development Notes

- Docker is intentionally not used in this project.
- Redis is reserved for future queue expansion; current functionality does not require it.
- The backend is JavaScript NestJS using Babel decorator support, not TypeScript source files.
- The frontend uses a modern Next.js version. Before changing Next-specific route behavior, consult local Next docs under `frontend/node_modules/next/dist/docs/` when available.

Footer: NexusCRM Developer Guide | Chapter 4 | Local Development Setup

Chapter 5

Environment Configuration

Chapter 5: Environment Configuration

Configuration Strategy

NexusCRM uses separate environment files:

- Backend: backend/.env
- Frontend: frontend/.env.local
- Reference: .env.example

Never commit real secrets.

Shared Runtime Variables

Variable	Purpose
NODE_ENV	Runtime mode
APP_DOMAIN	Backend tenant domain resolution
FRONTEND_URL	Backend link generation and CORS reference
APP_URL	Backend app URL fallback
CORS_ORIGIN	Allowed frontend origins

Frontend Variables

Variable	Purpose
NEXT_PUBLIC_APP_URL	Public frontend base URL
NEXT_PUBLIC_APP_DOMAIN	Domain used by frontend tenant routing
NEXT_PUBLIC_API_URL	Public backend URL
API_URL	Server-side Vercel proxy target
API_PROXY_TIMEOUT_MS	Proxy timeout for Render cold starts
NEXT_PUBLIC_WS_URL	Socket.IO backend URL
NEXT_PUBLIC_RECAPTCHA_SITE_KEY	reCAPTCHA browser key

Backend Variables

Variable	Purpose
PORT	API port, default 4000
MONGODB_URI	MongoDB Atlas or local URI
JWT_SECRET	JWT signing secret
JWT_EXPIRES_IN	JWT expiration, default style 7d
DNS_SERVERS	DNS resolver override; use system to disable custom resolvers
REQUEST_BODY_LIMIT	JSON/form payload limit, useful for attachments

Variable	Purpose
DEFAULT_TRIAL_DAYS	Default trial length for new tenants
LOCAL_FILE_STORAGE_DIR	Local file asset root
SEED_DEMO_DATA	Enables demo seed behavior

Superadmin Variables

Variable	Purpose
SUPERADMIN_EMAIL	Platform admin email
SUPERADMIN_PASSWORD	Platform admin password
SUPERADMIN_TENANT_SUBDOMAIN	System workspace subdomain

Email Variables

Variable	Purpose
BREVO_API_KEY	Transactional email API key
BREVO_SENDER_EMAIL	Verified sender
BREVO_SENDER_NAME	Sender display name
CONTACT_SALES_EMAIL	Destination for contact forms

Tenant SMTP/IMAP accounts are configured in the application UI rather than `.env`.

Google OAuth Variables

Variable	Purpose
GOOGLE_CLIENT_ID	Google OAuth client
GOOGLE_CLIENT_SECRET	Google OAuth secret
GOOGLE_REDIRECT_URI	OAuth callback URL

For backend routes with the global `/api` prefix, the callback is typically:

```
http://localhost:4000/api/auth/google/callback
```

Stripe Variables

Variable	Purpose
STRIPE_SECRET_KEY	Stripe test/live secret key
STRIPE_WEBHOOK_SECRET	Webhook signature secret
STRIPE_PRICE_PRO	Professional plan price id
STRIPE_PRICE_BUSINESS	Business plan price id
STRIPE_PRICE_ENTERPRISE	Enterprise plan price id

When Stripe is not configured, billing routes return mock URLs so developers can test UI flows without external billing.

Deployment-Specific Values

For Vercel frontend:

```
API_URL=https://your-render-backend.onrender.com
NEXT_PUBLIC_API_URL=https://your-render-backend.onrender.com
NEXT_PUBLIC_WS_URL=https://your-render-backend.onrender.com
API_PROXY_TIMEOUT_MS=55000
NEXT_PUBLIC_APP_URL=https://your-vercel-app.vercel.app
NEXT_PUBLIC_APP_DOMAIN=your-vercel-app.vercel.app
```

For Render backend:

```
FRONTEND_URL=https://your-vercel-app.vercel.app
CORS_ORIGIN=https://your-vercel-app.vercel.app
APP_DOMAIN=your-vercel-app.vercel.app
MONGODB_URI=mongodb+srv://...
JWT_SECRET=long-random-secret
```

Footer: NexusCRM Developer Guide | Chapter 5 | Environment Configuration

Chapter 6

Tenancy, Authentication, Sessions, And RBAC

Chapter 6: Tenancy, Authentication, Sessions, And RBAC

Tenancy Problem

A SaaS CRM must ensure that every customer workspace sees only its own records. Sales data, customer records, service tickets, emails, files, automation rules, and billing settings must never leak across tenants.

NexusCRM Tenancy Solution

NexusCRM uses a single MongoDB database with tenant-scoped documents. Tenant identity is resolved from:

- Host or forwarded host.
- Tenant path/subdomain.
- X-Tenant-Subdomain header.
- Custom domain stored on the Tenant model.

The backend then sets tenant context on the request. Global guards and interceptors enforce tenant membership and scope.

Tenant Model

The Tenant schema includes:

- name
- subdomain
- customDomain
- plan
- status
- settings
- defaultDepartmentId
- stripeCustomerId
- stripeSubscriptionId
- billingPeriodEnd
- trialEndsAt
- onboardingCompleted

Authentication Flow

Signup:

- User submits email, password, company/workspace name, subdomain, and plan.
- Backend creates tenant with trial status.
- Backend seeds onboarding defaults.
- Backend creates owner user.
- Backend creates UserTenant membership with owner role.
- Verification email or OTP is sent when configured.
- Trial notification is created.

Login:

- User submits email/password and tenant id or subdomain.
- Backend validates credentials.
- Backend verifies tenant status and membership.
- Backend returns JWT, tenant profile, role, and CASL rules.
- Frontend stores token and tenant context in localStorage.

Session restore:

- Frontend calls `/api/auth/me`.
- Backend returns current user, tenant, permissions, and rules.
- Frontend refreshes session state and redirects valid sessions away from login/signup pages.
- Stale sessions are cleared.

Frontend Session Storage

The central client is `frontend/lib/api.js`.

Stored keys:

- `crm_token`
- `crm_tenant`
- `crm_tenant_id`
- `crm_rules`
- `crm_is_superadmin`

TenantGate

`frontend/components/layout/tenant-gate.js` protects tenant pages.

Responsibilities:

- Validate token.
- Fetch `/auth/me`.
- Update tenant/session/rules.
- Block superadmin from tenant workspace routes.
- Redirect unauthenticated users to login.
- Correct tenant mismatch.
- Enforce onboarding route behavior.

RBAC Model

Roles include:

- `owner`
- `admin`
- `chief`
- `manager`

- operator
- co-worker
- accountant
- task_operator

Permissions are built from:

- Default role templates.
- Group permissions.
- Member-level permissions.

CASL rules are returned to the frontend and used by:

- components/can.js
- lib/ability.js
- Navigation visibility.
- Settings and sensitive action gates.

Backend Security Stack

Layer	Purpose
TenantGuard	Validates JWT, tenant, tenant status, and membership
RolesGuard	Checks role decorators
PoliciesGuard	Applies CASL policy handlers
@Public()	Allows public endpoints
@SuperadminOnly()	Restricts platform administration
TenantScopeInterceptor	Helps scope tenant-aware responses
Policy handlers	Reusable checks such as canManageSettings, canManageInbox, canManageSecurity

Developer Rule

Frontend permission gating is not enough. Any sensitive backend route must also use appropriate guards and policy handlers.

Footer: NexusCRM Developer Guide | Chapter 6 | Tenancy, Authentication, Sessions, And RBAC

Chapter 7

Backend Engineering Guide

Chapter 7: Backend Engineering Guide

Backend Stack

- NestJS 11.
- JavaScript with Babel decorator support.
- CommonJS modules.
- Mongoose.
- Passport JWT.
- CASL permissions.
- Socket.IO.
- Stripe.
- Nodemailer/Brevo.
- ExcelJS and PDFKit.

Backend Module Pattern

Most backend features follow this structure:

```
modules/{feature}/  
  ■■■ {feature}.module.js  
  ■■■ {feature}.controller.js  
  ■■■ {feature}.service.js  
  ■■■ schemas/  
    ■■■ {entity}.schema.js
```

Controller Responsibilities

Controllers should:

- Declare route prefix with `@Controller()`.
- Bind request data through `@Bind()`.
- Use guards and policies for sensitive routes.
- Delegate business logic to services.
- Avoid direct database operations when service methods exist.

Service Responsibilities

Services should:

- Validate business rules.
- Apply tenant filters.
- Perform aggregation and transformations.
- Record activities where important.
- Create notifications for user-facing events.
- Keep persistence details centralized.

Model Injection Pattern

Because the backend uses JavaScript NestJS with Babel decorators, model injection uses `withModels()` from:

```
backend/src/common/providers/with-models.js
```

Example concept:

```
withModels(SomeService, {  
  someModel: 'SomeModel',  
});
```

This creates the service and assigns injected Mongoose models to instance properties.

Schema Guidelines

Tenant-scoped schemas should:

- Include `tenantId`.
- Index `tenantId`.
- Add compound indexes for common list queries.
- Store references as `Schema.Types.ObjectId`.
- Avoid unbounded embedded arrays unless the data is naturally small.

Activity Recording

Activity is handled by:

- `backend/src/modules/activity/activity.service.js`
- `backend/src/modules/activity/activity-recorder.js`
- `ActivityEvent` schema

Use activity events for:

- Security events.
- File operations.
- Ticket actions.
- CRM changes.
- Automation outcomes.
- Import/export jobs.

Error Handling

Use Nest exceptions:

- `BadRequestException`
- `UnauthorizedException`
- `ForbiddenException`
- `NotFoundException`
- `ConflictException`

Do not return raw error objects to clients.

Backend Feature Modules

Key modules:

- **auth**: authentication, verification, password reset, profile, superadmin bootstrap.
- **tenant**: tenant settings, onboarding, tenant resolution, lead sources.
- **rbac**: groups, departments, invitations, membership, permissions.
- **crm**: leads, deals, companies, contacts, requests, customer 360.
- **tasks**: tasks, projects, memos.
- **mail**: email accounts, massmail, shared inbox, Gmail OAuth.
- **realtime**: chat and notifications.
- **subscription**: plans, billing summary, Stripe checkout, Stripe portal, webhooks.
- **files**: file asset creation, download, and metadata.
- **data-jobs**: import/export jobs and artifacts.
- **jobs**: worker lease and job health.
- **extensions**: products, sales documents, tickets, automation, report exports, live chat, SMS, knowledge.
- **security**: security overview, policy, audit event stream, audit export.
- **superadmin**: platform tenant administration.

Footer: NexusCRM Developer Guide | Chapter 7 | Backend Engineering Guide

Chapter 8

Frontend Engineering Guide

Chapter 8: Frontend Engineering Guide

Frontend Stack

- Next.js 16 App Router.
- React 19.
- Tailwind CSS 4.
- React Query.
- CASL permissions.
- Socket.IO client.
- Lucide icons.
- Shared in-house UI primitives.

Route Organization

```
frontend/app/  
  (auth)/  
  [tenant]/  
    api/  
    superadmin/  
    layout.js  
    globals.css  
    icon.svg
```

Root Layout

frontend/app/layout.js:

- Loads font.
- Applies global theme script.
- Wraps the app in providers.
- Renders toast stack through NotificationProvider.

Tenant Layout

frontend/app/[tenant]/layout.js:

- Wraps workspace pages in TenantGate.
- Provides React Query.
- Provides session context.
- Renders AppShell.

App Shell

frontend/components/layout/app-shell.js:

- Renders sidebar.
- Renders topbar.
- Displays trial banner when tenant status is trial.

- Provides the main workspace frame.

Navigation

`frontend/lib/navigation.js` defines workspace navigation.

Navigation entries can include:

- `href`
- `label`
- `icon`
- `action`
- `subject`

The `action` and `subject` values are used to hide items the user cannot access.

API Helper Pattern

The central JSON client is:

```
frontend/lib/api.js
```

Feature-specific API helpers live in:

- `crm-api.js`
- `extensions-api.js`
- `mail-api.js`
- `tasks-api.js`
- `analytics-api.js`
- `data-jobs-api.js`
- `files-api.js`
- `metadata-api.js`
- `realtime-api.js`
- `security-api.js`

API Proxy

Browser requests are same-origin:

```
/api/*
```

The Next.js route proxy in `frontend/app/api/[...path]/route.js` forwards them to:

- `API_URL`
- `NEXT_PUBLIC_API_URL`
- local fallback `http://127.0.0.1:4000`

The proxy supports:

- JSON requests.
- Binary responses.
- Render cold-start timeout messages.

- Production misconfiguration warnings.

UI Conventions

Use shared primitives:

- `components/ui/*`
- `components/settings/settings-layout.js`
- `components/crm/CrmListPage.js`
- `components/crm/ModuleListPage.js`
- `components/chat/ObjectChat.js`
- `components/notifications/NotificationsDropdown.js`

Styling Conventions

Global tokens are in:

`frontend/app/globals.css`

Prefer existing tokens and utility classes:

- `bg-background`
- `bg-card`
- `bg-control`
- `border-border`
- `text-foreground`
- `text-muted`
- `text-muted-foreground`
- `text-brand`
- `focus-ring`
- `input-base`

Frontend Developer Rules

- Do not duplicate API fetch logic inside pages.
- Do not hardcode tenant URLs when `getTenantUrl()` exists.
- Use `Can` for permission-aware UI.
- Keep settings pages inside `SettingsPageShell`.
- Use shared list components for CRM and extension entities.
- Handle loading and error states on every asynchronous page.

Footer: NexusCRM Developer Guide | Chapter 8 | Frontend Engineering Guide

Chapter 9

Module-By-Module Functional Guide

Chapter 9: Module-By-Module Functional Guide

Authentication Module

Problem:

Users need secure tenant-scoped login, signup, invitation acceptance, password reset, email verification, session restore, and tenant switching.

Solution:

- Backend module: `backend/src/modules/auth`
- Frontend routes: `/login`, `/signup`, `/register`, `/forgot-password`, `/reset-password`, `/verify-email`, `/invite/[token]`
- Session restore through `/api/auth/me`
- JWT authentication with tenant membership validation
- Superadmin login and platform access

Key functionality:

- Workspace registration.
- Email verification and OTP.
- Tenant discovery for users belonging to multiple tenants.
- Tenant switching.
- Password reset.
- Profile update.
- Security activity events.

Tenant And Settings Module

Problem:

Each workspace requires identity, company settings, onboarding, subdomain/custom domain configuration, departments, lead sources, and workspace defaults.

Solution:

- Backend module: `backend/src/modules/tenant`
- Frontend routes: `/[tenant]/settings`, `/settings/tenant`, `/settings/departments`, `/settings/email`, `/onboarding`

Key functionality:

- Tenant resolution.
- Onboarding status and completion.
- Company settings.
- Default department.
- Currency, timezone, language, tax settings.
- Lead source configuration.
- Trial end date and status.

RBAC Module

Problem:

Enterprise CRM users require different access levels by role, department, group, and explicit permission.

Solution:

- Backend modules: `backend/src/modules/rbac`, `backend/src/common/casl`, `backend/src/common/policies`
- Frontend helpers: `frontend/lib/ability.js`, `frontend/components/can.js`

Key functionality:

- Users and memberships.
- Invitations.
- Departments.
- Groups and permissions.
- Default role permission templates.
- CASL rule generation.
- Frontend route/action visibility.

CRM Module

Problem:

Sales and customer teams require tenant-scoped records for leads, contacts, companies, deals, and requests.

Solution:

- Backend module: `backend/src/modules/crm`
- Frontend CRM routes under `/[tenant]/crm`

Key functionality:

- Leads with dedupe, routing rules, conversion, and assignment.
- Companies with customer 360 and account planning.
- Contacts as customer/person records.
- Deals with pipeline, payments, attachments, history, emails, and line items.
- Requests for inbound customer or public form submissions.
- Custom fields through metadata.
- Activity timeline.

Sales Module

Problem:

Sales teams need products, quotes, orders, invoices, line items, PDF documents, and conversion paths.

Solution:

- Backend module: `backend/src/modules/extensions`
- Shared service: sales document service
- Frontend routes under `/[tenant]/sales`

Key functionality:

- Product catalog.
- Opportunity line items.
- Quotations.
- Orders.
- Invoices.
- Line item editing.
- Quote-to-order and quote-to-invoice conversions.
- Branded PDF generation.
- Sales pipeline view.

Service Desk Module

Problem:

Support teams need ticket routing, queues, macros, replies, notes, SLA fields, and conversation history.

Solution:

- Backend ticket service/controller under `backend/src/modules/extensions`
- Frontend routes under `/[tenant]/service`

Key functionality:

- Tickets.
- Ticket queues.
- Ticket macros.
- Public replies.
- Internal notes.
- Macro application.
- Assignment and routing.
- Resolve/reopen workflows.
- SLA state.
- Activity recording.
- Object chat inside ticket detail.

Mail And Shared Inbox Module

Problem:

Teams need outbound email, tenant-specific SMTP/IMAP accounts, Gmail OAuth, mass mail, and shared inbox triage.

Solution:

- Backend module: `backend/src/modules/mail`
- Frontend helper: `frontend/lib/mail-api.js`
- Frontend routes: `/inbox`, `/massmail`, `/marketing/campaigns`, `/settings/email-accounts`

Key functionality:

- Email account management.

- Account testing.
- IMAP sync.
- Gmail OAuth connection.
- Shared inbox threads and messages.
- Reply and internal note actions.
- Thread assignment, read state, archive, and entity linking.
- Mass mail campaigns and unsubscribe handling.

Tasks, Projects, And Memos

Problem:

CRM work needs follow-up execution, internal notes, tasks, projects, review, and conversion flows.

Solution:

- Backend module: `backend/src/modules/tasks`
- Frontend routes: `/tasks`, `/projects`, `/projects/[id]`, `/memos`

Key functionality:

- Task creation and assignment.
- Subtasks.
- Comments.
- Hide-for-me behavior.
- Projects and project detail.
- Memos, reviews, and conversion to tasks/projects.
- Notifications for assigned or relevant work.

Automation Module

Problem:

Teams need executable workflows that respond to CRM/service events with conditions, actions, retries, and logs.

Solution:

- Backend extension schemas: `AutomationRule`, `AutomationRun`
- Backend runtime logic in extension configs.
- Frontend route: `/[tenant]/automation`

Key functionality:

- Automation rules.
- Trigger configuration.
- Conditions and condition modes.
- Actions such as notifications, task creation, ticket creation, record update, owner assignment, tags, macros, and webhooks.
- Dry-run and execution history.
- Retry policy and retry runs.

Reports, Analytics, And Data Jobs

Problem:

Managers need dashboards, reports, exports, import jobs, audit exports, and operational visibility.

Solution:

- Backend modules: analytics, dashboard, data-jobs, jobs, security
- Frontend routes: /dashboard, /analytics, /reports, /settings/data-jobs, /settings/audit, /settings/security

Key functionality:

- KPI widgets.
- Sales trend.
- Income summaries.
- Sales funnel.
- Team performance.
- Report exports.
- Data imports and exports.
- Job preview/run/retry/cancel.
- Worker lease and job health.
- Security event listing and audit export.

Files Module

Problem:

Data jobs, chat, exports, and user workflows require file persistence, downloads, and metadata.

Solution:

- Backend module: backend/src/modules/files
- Frontend helper: frontend/lib/files-api.js

Key functionality:

- Create file asset from base64/content.
- Store file metadata.
- Download file asset.
- Soft delete file asset.
- Record file activity.
- Notify uploader.

Realtime And Notifications Module

Problem:

Enterprise users need immediate awareness of messages, assignments, trial alerts, billing changes, uploads, and operational actions.

Solution:

- Backend module: `backend/src/modules/realtime`
- Socket hub: `backend/src/realtime/socket-hub.js`
- Frontend components: `ObjectChat`, `NotificationsDropdown`

Key functionality:

- Socket JWT authentication.
- User rooms.
- Object rooms.
- Typing events.
- Chat messages.
- Attachments.
- Read receipts.
- Persistent notifications.
- Toast display.

Billing And Subscription Module

Problem:

NexusCRM must operate as a SaaS product with plans, pricing, trials, usage limits, checkout, portal, and invoices.

Solution:

- Backend module: `backend/src/modules/subscription`
- Frontend route: `/[tenant]/settings/billing`

Key functionality:

- Canonical plans: Starter, Professional, Business, Enterprise.
- Plan limits.
- Trial countdown.
- Billing summary.
- Stripe checkout.
- Stripe portal.
- Stripe webhook handling.
- Mock billing flow when Stripe keys are not configured.
- In-app billing notifications.

Superadmin Module

Problem:

Platform operators require cross-tenant visibility and tenant administration.

Solution:

- Backend module: `backend/src/modules/superadmin`
- Frontend routes under `/superadmin`

Key functionality:

- Tenant list.
- Tenant detail.
- Suspend/activate tenant.
- Change tenant plan.
- Platform stats.
- Platform settings.
- Superadmin-only authentication context.

Footer: NexusCRM Developer Guide | Chapter 9 | Module-By-Module Functional Guide

Chapter 10

API Route Reference

Chapter 10: API Route Reference

All backend HTTP routes are prefixed with:

/api

Auth And Session Routes

Method	Route	Purpose
POST	/auth/register	Create tenant and owner
POST	/auth/signup	Signup flow with plan mapping
POST	/auth/login	Tenant-scoped login
POST	/auth/superadmin/login	Platform admin login
POST	/auth/discover-tenants	Find workspaces for credentials
POST	/auth/accept-invite	Accept invite token
POST	/auth/verify-email	Verify email token
POST	/auth/verify-otp	Verify OTP
POST	/auth/resend-verification	Resend verification
POST	/auth/forgot-password	Start password reset
POST	/auth/reset-password	Complete password reset
GET	/auth/my-tenants	List current user's tenants
POST	/auth/switch-tenant	Switch active tenant JWT
GET	/auth/me	Current profile, tenant, permissions, rules
PUT	/users/profile	Update profile

Tenant And RBAC Routes

Method	Route	Purpose
GET	/tenants/resolve/:subdomain	Resolve tenant metadata
GET	/tenants	Superadmin tenant list
GET	/tenant-data/members	Tenant member list
GET	/tenants/settings	Tenant settings
PUT	/tenants/settings	Update tenant settings
GET	/tenants/onboarding/status	Onboarding status
POST	/tenants/onboarding/complete	Complete onboarding
GET	/tenants/users	Members
POST	/tenants/invite	Invite member
PATCH	/tenants/users/:memberId	Update member
POST	/tenants/users/:memberId/remove	Remove member
GET	/tenants/groups	Groups and permissions
PATCH	/tenants/groups/:groupId	Update group permissions

Method	Route	Purpose
GET	/tenants/departments	Departments
POST	/tenants/departments	Create department
PATCH	/tenants/departments/:departmentId	Update department
GET	/tenants/lead-sources	Lead sources
PATCH	/tenants/lead-sources/:id	Update lead source

CRM Routes

Method	Route	Purpose
GET/POST	/leads	List/create leads
GET/PATCH/DELETE	/leads/:id	Lead detail/update/delete
POST	/leads/bulk	Bulk lead actions
GET/POST	/leads/routing-rules	Lead routing rules
PATCH/DELETE	/leads/routing-rules/:ruleId	Update/delete routing rule
GET	/leads/:id/duplicates	Duplicate detection
POST	/leads/:id/convert	Convert lead
POST	/leads/:id/route	Route lead
GET/POST	/companies	List/create companies
GET/PATCH/DELETE	/companies/:id	Company detail/update/delete
GET	/companies/:id/360	Customer 360 aggregation
PATCH	/companies/:id/account-plan	Account plan update
GET/POST	/contacts	Generated contact CRUD
GET/POST	/requests	Generated request CRUD
GET/POST	/deals	List/create deals
GET/PATCH	/deals/:id	Deal detail/update
POST	/deals/bulk	Bulk deal actions
GET/POST	/deals/pipelines	Deal pipelines
PATCH	/deals/pipelines/:pipelineId	Update pipeline
GET/POST	/deals/:id/line-items	Deal line items
PATCH/DELETE	/deals/:id/line-items/:lineItemId	Update/delete line item
GET/POST	/deals/:id/emails	Deal emails
GET/POST	/deals/:id/payments	Deal payments
GET	/deals/:id/attachments	Deal attachments
GET	/deals/:id/history	Deal audit history

Task And Work Routes

Method	Route	Purpose
GET/POST	/tasks	List/create tasks
GET/PATCH	/tasks/:id	Task detail/update
POST	/tasks/:id/subtasks	Add subtask
POST	/tasks/:id/comments	Add comment
POST	/tasks/:id/hide-for-me	Hide task for current user
GET/POST	/projects	List/create projects
GET	/projects/:id	Project detail
GET/POST	/memos	List/create memos
GET/PATCH	/memos/:id	Memo detail/update
POST	/memos/:id/review	Review memo
POST	/memos/:id/convert-to-task	Convert memo to task
POST	/memos/:id/convert-to-project	Convert memo to project

Sales Document Routes

The following route groups support list/create/detail/update/delete plus document-specific actions:

- /quotations
- /orders
- /invoices

Additional routes:

Method	Route Pattern	Purpose
GET/POST	/:doc/:id/line-items	List/add line items
PATCH/DELETE	/:doc/:id/line-items/:lineItemId	Update/delete line item
POST	/:doc/:id/convert-to-order	Convert to order
POST	/:doc/:id/convert-to-invoice	Convert to invoice
GET	/:doc/:id/pdf	Download branded PDF

Service Desk Routes

Method	Route	Purpose
GET/POST	/tickets	List/create tickets
GET/PATCH/DELETE	/tickets/:id	Ticket detail/update/delete
GET	/tickets/:id/conversation	Conversation
POST	/tickets/:id/replies	Public reply
POST	/tickets/:id/notes	Internal note
POST	/tickets/:id/apply-macro	Apply macro
POST	/tickets/:id/route	Route ticket
POST	/tickets/:id/resolve	Resolve ticket

Method	Route	Purpose
POST	/tickets/:id/reopen	Reopen ticket
GET/POST	/ticket-queues	Queue CRUD
GET/POST	/ticket-macros	Macro CRUD

Mail And Inbox Routes

Method	Route	Purpose
GET/POST	/email-accounts	Email accounts
POST	/email-accounts/:id/test	Test account
POST	/email-accounts/sync	Sync IMAP/shared inbox
POST	/emails/send	Send email
GET/POST	/massmail/campaigns	Campaigns
GET	/massmail/campaigns/:id	Campaign detail
POST	/massmail/campaigns/:id/send	Send campaign
POST	/massmail/preview-recipients	Preview recipients
GET	/massmail/templates	Templates
GET	/massmail/unsubscribes	Unsubscribes
POST	/unsubscribe	Public unsubscribe
GET	/inbox/threads	Shared inbox threads
GET	/inbox/threads/:id	Thread detail
POST	/inbox/sync	Sync inbox
POST	/inbox/threads/:id/reply	Reply
POST	/inbox/threads/:id/notes	Internal note
PATCH	/inbox/threads/:id/assign	Assign
PATCH	/inbox/threads/:id/read	Read state
PATCH	/inbox/threads/:id/archive	Archive
PATCH	/inbox/threads/:id/link	Link entity

Realtime, Files, Jobs, Billing, Security

Method	Route	Purpose
GET/POST	/chat/messages	List/create object chat messages
PATCH	/chat/messages/:id/read	Mark chat message read
GET	/chat/attachments/:id/download	Download chat attachment
GET	/notifications	List notifications
GET	/notifications/unread-count	Unread count
PATCH	/notifications/:id/read	Mark notification read
GET/POST	/files	List/create file assets

Method	Route	Purpose
GET	/files/:id/download	Download file asset
DELETE	/files/:id	Delete file asset
GET/POST	/data-jobs	List/create jobs
GET	/data-jobs/:id	Job detail
POST	/data-jobs/:id/preview	Preview import
POST	/data-jobs/:id/run	Run job
POST	/data-jobs/:id/retry	Retry job
POST	/data-jobs/:id/cancel	Cancel job
GET	/jobs/health	Job health
POST	/jobs/lease-next	Lease next job
GET	/billing	Billing summary
POST	/billing/portal	Stripe portal
POST	/billing/checkout	Stripe checkout
POST	/billing/webhook	Stripe webhook
GET	/security/overview	Security overview
PATCH	/security/policy	Update security policy
GET	/security/events	Audit/security events
POST	/security/audit-export	Queue audit export

Generated Extension Routes

Generated guarded routes exist for:

- /products
- /ticket-queues
- /ticket-macros
- /sms
- /knowledge
- /automation
- /automation-runs
- /report-export-jobs
- /live-chat

Generated controllers generally support:

- GET /
- POST /
- GET /:id
- PATCH /:id
- DELETE /:id
- GET /:id/download

- POST `/:id/run`
- POST `/bulk`

Footer: NexusCRM Developer Guide | Chapter 10 | API Route Reference

Chapter 11

Data Model And Persistence

Chapter 11: Data Model And Persistence

Persistence Strategy

NexusCRM uses MongoDB through Mongoose. The application uses a single database and tenant-scoped documents. This is simpler to deploy than one-database-per-tenant while still providing strong logical isolation when guards and query patterns are followed.

Tenant-Scoped Data Rule

Every customer-owned model should include:

```
tenantId: { type: Schema.Types.ObjectId, ref: 'Tenant', required: true, index: true }
```

Queries should include:

```
{ tenantId, ...otherFilters }
```

Key Schemas

Identity and tenancy:

- Tenant
- User
- UserTenant
- Group
- Department
- Invitation

CRM:

- Lead
- LeadRoutingRule
- Contact
- Company
- Deal
- DealPipeline
- Request
- CrmEmail
- Attachment
- AuditLog

Work:

- Task
- Project
- Memo

Operations:

- ActivityEvent
- DataJob

- FileAsset
- Notification
- ChatMessage

Mail:

- EmailAccount
- MassmailCampaign
- MailboxThread
- MailboxMessage
- Unsubscribe

Sales and service extensions:

- Product
- Quotation
- Order
- Invoice
- Ticket
- TicketQueue
- TicketMacro
- AutomationRule
- AutomationRun
- ReportExportJob
- LiveChatSession
- KnowledgeArticle
- SmsCampaign

Embedded Versus Referenced Data

Use embedded data for:

- Short line item arrays.
- Message attachment metadata.
- Ticket conversation entries.
- Snapshot details that should not change after creation.

Use references for:

- Users.
- Tenants.
- Companies.
- Contacts.
- Deals.
- Files.
- Owners/assignees.

Activity Events

ActivityEvent records important user and system activity:

- Action.
- Source.
- Severity.
- Entity type/id/name.
- Actor.
- Summary.
- Visibility.
- Metadata.

File Assets

FileAsset stores:

- Filename.
- Original name.
- MIME type.
- Size.
- Storage provider.
- Storage key.
- Entity type/id.
- Purpose.
- Checksum.
- Status.
- Uploaded by.
- Expiry.
- Metadata.

Data Jobs

DataJob supports:

- Import/export/report/sync/enrichment style jobs.
- Progress.
- Logs.
- Source/result/error file ids.
- Preview rows.
- Attempts and leasing.
- Rollback and committed record ids.

Schema Evolution Guidance

When adding fields:

- Keep backwards compatibility for existing persisted documents.
- Provide defaults.
- Index only fields used in common queries.
- Avoid changing enum values without migration logic.
- Make frontend null-safe for optional fields.

Footer: NexusCRM Developer Guide | Chapter 11 | Data Model And Persistence

Chapter 12

Realtime, Chat, Files, And Notifications

Chapter 12: Realtime, Chat, Files, And Notifications

Realtime Problem Statement

CRM users need to know when teammates send messages, upload files, read conversations, assign work, update billing state, or trigger important workflow events. Polling every page is inefficient and gives poor user experience.

Realtime Solution

NexusCRM uses:

- Socket.IO server in `backend/src/realtime/socket-hub.js`.
- Socket.IO client in `frontend/lib/socket.js`.
- Persistent notifications in MongoDB.
- Toast display in the frontend.

Socket Authentication

The client sends JWT through Socket.IO auth. The backend verifies the token using the JWT secret and attaches:

- User id.
- Tenant id.
- User name.

Socket Rooms

User room:

```
tenant:{tenantId}:user:{userId}
```

Object room:

```
tenant:{tenantId}:object:{entityType}:{entityId}
```

User rooms receive personal notifications. Object rooms receive chat, typing, and read receipt events.

Object Chat

Frontend component:

```
frontend/components/chat/ObjectChat.js
```

Backend:

```
backend/src/modules/realtime/chat.controller.js  
backend/src/modules/realtime/chat.service.js  
backend/src/modules/realtime/schemas/chat-message.schema.js
```

Functionality:

- Message list.
- Send message.
- Typing indicator.
- Attach files.
- Emoji insertion.

- Download attachments.
- Read receipts.
- Realtime message updates.
- Error handling.

Chat Attachment Flow

Current chat attachments are sent with encoded payload metadata and saved under the backend chat upload area. The message stores attachment metadata such as:

- Attachment id.
- File asset id when available.
- Filename.
- MIME type.
- Size.
- Download URL.

The backend enforces a size limit and the frontend rejects files over the configured attachment limit before sending.

Read Receipts

ChatMessage stores readBy entries:

- userId
- userName
- readAt

The frontend calls:

```
PATCH /api/chat/messages/:id/read
```

The backend emits:

```
message:read
```

Notifications

Backend notification service:

```
backend/src/modules/realtime/notifications.service.js
```

Frontend notification dropdown:

```
frontend/components/notifications/NotificationsDropdown.js
```

Notification fields:

- Type.
- Title.
- Body.
- Href.
- Entity type.
- Entity id.
- Read state.

- Created time.

Notification Sources

Current in-app notifications include:

- Trial started.
- Trial ending/expired reminder.
- Billing portal/checkout events.
- Stripe subscription updates.
- Chat messages.
- File uploaded events.

Files Module

Backend:

```
backend/src/modules/files
```

Frontend:

```
frontend/lib/files-api.js
```

Functionality:

- List files.
- Create file from base64/content.
- Download files.
- Soft delete files.
- Record activity.
- Notify uploader.

Developer Guidance

Add notifications when an action:

- Requires user attention.
- Affects billing or security.
- Assigns work.
- Completes or fails a background operation.
- Creates customer-facing communication.
- Changes ownership, status, or SLA state.

Footer: NexusCRM Developer Guide | Chapter 12 | Realtime, Chat, Files, And Notifications

Chapter 13

Billing, Plans, Trials, And SaaS Operations

Chapter 13: Billing, Plans, Trials, And SaaS Operations

SaaS Billing Problem Statement

A CRM SaaS product must define plan limits, handle trials, route users to checkout, expose billing status, and keep administrators aware of subscription state.

Canonical Plans

Backend constants live in:

```
backend/src/common/constants/plans.js
```

Plans:

- Starter
- Professional
- Business
- Enterprise

Legacy aliases:

- Free -> Starter
- Pro -> Professional

Plan Limits

Plan limits include:

- Users.
- Storage MB.
- Deals.
- Emails per month.

Enterprise uses -1 for unlimited values.

Public Plans

Public plan metadata is served by:

```
GET /api/public/plans
```

Frontend fallback pricing lives in:

```
frontend/lib/plans.js
```

Fallback data should match backend plan ids and names.

Billing Summary

Route:

```
GET /api/billing
```

Returns:

- Current plan.

- Tenant status.
- Limits.
- Usage.
- Stripe customer id.
- Billing period end.
- Trial end date.
- Trial days remaining.
- Available plans.
- Invoices.

Trial Behavior

New tenants receive:

- `status: trial`
- `trialEndsAt`

Trial length defaults to:

```
DEFAULT_TRIAL_DAYS=7
```

Trial reminders are created when billing is viewed near trial expiration.

Checkout

Route:

```
POST /api/billing/checkout
```

Payload:

```
{
  "plan": "Professional",
  "returnUrl": "https://app.example.com/acme/settings/billing"
}
```

If Stripe is configured, a real checkout URL is returned. If not, a mock checkout URL is returned.

Portal

Route:

```
POST /api/billing/portal
```

Returns a Stripe billing portal session or a mock URL.

Webhook

Route:

```
POST /api/billing/webhook
```

The webhook requires raw body parsing for signature verification. Do not add normal JSON parsing before the webhook route.

Billing Page

Frontend page:

frontend/app/[tenant]/settings/billing/page.js

Displays:

- Current plan.
- Trial warning.
- Usage limits.
- Upgrade path.
- Plan pricing cards.
- Billing portal button.
- Invoices.

Footer: NexusCRM Developer Guide | Chapter 13 | Billing, Plans, Trials, And SaaS Operations

Chapter 14

Integrations And External Services

Chapter 14: Integrations And External Services

MongoDB

Required for all application data.

Environment:

```
MONGODB_URI=mongodb+srv://...
```

Test:

- Backend starts.
- `/api/health` returns OK.
- Signup creates tenant/user records.

Brevo

Used for transactional email:

- Signup verification.
- OTP.
- Invitations.
- System emails.

Environment:

```
BREVO_API_KEY=  
BREVO_SENDER_EMAIL=  
BREVO_SENDER_NAME=NexusCRM
```

SMTP/IMAP

Configured per tenant in the application UI.

Used for:

- Sending deal emails.
- Mass mail.
- IMAP/shared inbox sync.

Google OAuth/Gmail

Used for Gmail connection in settings.

Environment:

```
GOOGLE_CLIENT_ID=  
GOOGLE_CLIENT_SECRET=  
GOOGLE_REDIRECT_URI=http://localhost:4000/api/auth/google/callback
```

Stripe

Used for:

- Checkout.
- Billing portal.

- Subscription webhooks.
- Invoice history.

Environment:

```
STRIPE_SECRET_KEY=  
STRIPE_WEBHOOK_SECRET=  
STRIPE_PRICE_PRO=  
STRIPE_PRICE_BUSINESS=  
STRIPE_PRICE_ENTERPRISE=
```

reCAPTCHA

Used for signup and public form hardening when configured.

Environment:

```
RECAPTCHA_SECRET_KEY=  
NEXT_PUBLIC_RECAPTCHA_SITE_KEY=
```

Zadarma VoIP

Optional click-to-call integration. If unset, phone links can fall back to normal tel: links.

Environment:

```
ZADARMA_KEY=  
ZADARMA_SECRET=  
ZADARMA_FROM_NUMBER=
```

Socket.IO

Built into the backend. Used for:

- Chat.
- Typing indicators.
- Read receipts.
- In-app notifications.

Frontend variable:

```
NEXT_PUBLIC_WS_URL=https://your-backend.onrender.com
```

Integration Test Order

- MongoDB and JWT.
- Signup/login/tenant workspace.
- Demo CRM data.
- Brevo verification/invite emails.
- SMTP or Gmail OAuth.
- Stripe test checkout and webhook.
- reCAPTCHA.
- Zadarma.
- Socket.IO chat in two browsers.

Footer: NexusCRM Developer Guide | Chapter 14 | Integrations And External Services

Chapter 15

Deployment Guide

Chapter 15: Deployment Guide

Deployment Architecture

Service	Platform	Directory
Frontend	Vercel	frontend/
Backend	Render	backend/
Database	MongoDB Atlas	external
Email	Brevo	external
Billing	Stripe	external

Backend On Render

Render settings:

- Root directory: backend
- Build command: `npm install`
- Start command: `npm run start:prod`
- Health check: `/api/health`

Required env:

```
PORT=4000
MONGODB_URI=mongodb+srv://...
JWT_SECRET=long-random-secret
FRONTEND_URL=https://your-app.vercel.app
CORS_ORIGIN=https://your-app.vercel.app
APP_DOMAIN=your-app.vercel.app
SUPERADMIN_EMAIL=admin@example.com
SUPERADMIN_PASSWORD=strong-password
SUPERADMIN_TENANT_SUBDOMAIN=system
```

Recommended env:

```
REQUEST_BODY_LIMIT=12mb
DEFAULT_TRIAL_DAYS=7
DNS_SERVERS=1.1.1.1,8.8.8.8
```

Frontend On Vercel

Vercel settings:

- Root directory: frontend
- Framework: Next.js
- Build: `npm run build`

Required env:

```
NEXT_PUBLIC_APP_URL=https://your-app.vercel.app
NEXT_PUBLIC_APP_DOMAIN=your-app.vercel.app
API_URL=https://your-render-backend.onrender.com
NEXT_PUBLIC_API_URL=https://your-render-backend.onrender.com
NEXT_PUBLIC_WS_URL=https://your-render-backend.onrender.com
API_PROXY_TIMEOUT_MS=55000
```

Use the backend root URL without `/api`.

Render Cold Starts

Render free or low-cost services can cold start. NexusCRM's Vercel proxy has a longer timeout and returns a clear message when the backend does not respond fast enough.

If users see timeout messages:

- Confirm Render service is awake.
- Check `/api/health`.
- Confirm Vercel `API_URL` is the Render backend root.
- Confirm Render `CORS_ORIGIN` includes the Vercel URL.

Post-Deploy Test

- Open frontend.
- Open backend `/api/health`.
- Signup a tenant.
- Verify email if Brevo is configured.
- Login.
- Visit dashboard.
- Open settings billing.
- Send chat message.
- Check notification bell.
- Test superadmin route.

Footer: NexusCRM Developer Guide | Chapter 15 | Deployment Guide

Chapter 16

Validation, QA, And Smoke Testing

Chapter 16: Validation, QA, And Smoke Testing

Frontend Validation

Run:

```
npm run lint --prefix frontend
npm run build --prefix frontend
```

Pass criteria:

- ESLint exits with code 0.
- Next build compiles successfully.
- Static and dynamic routes are generated.
- No missing imports.
- No invalid hook warnings introduced by edited code.

Backend Smoke Validation

Run:

```
npm run start --prefix backend
```

Pass criteria:

- MongoDB connects.
- Nest module graph initializes.
- Routes map successfully.
- Superadmin sync does not fail.
- Server prints API URL.
- `/api/health` returns OK.

Health test:

```
node -e "fetch('http://127.0.0.1:4000/api/health').then(async r=>{console.log(r.status); console.log(await r.text())})"
```

Feature Smoke Tests

Authentication:

- Signup.
- Verify email/OTP if configured.
- Login.
- Session restore.
- Tenant switching.
- Logout/stale token behavior.

RBAC:

- Owner can access settings.
- Co-worker cannot see restricted admin modules.
- Backend blocks restricted sensitive routes.

CRM:

- Create lead.
- Detect duplicates.
- Convert lead.
- Create deal.
- Add deal line item.
- View company 360.

Sales:

- Create product.
- Create quotation.
- Add line items.
- Convert to order/invoice.
- Download PDF.

Service:

- Create ticket.
- Route ticket.
- Add reply.
- Add internal note.
- Apply macro.
- Resolve/reopen.

Realtime:

- Open same record in two browsers.
- Send chat message.
- Attach file.
- Insert emoji.
- Confirm read receipt.
- Confirm notification toast/dropdown.

Billing:

- View billing summary.
- Confirm trial banner.
- Start checkout.
- Use mock flow when Stripe is absent.
- Use Stripe test flow when keys exist.

Data jobs:

- Create import job.
- Preview.
- Run.
- Download artifacts.
- Retry/cancel.

Regression Risk Areas

High-risk changes usually touch:

- TenantGuard
- Auth/session response shape
- `frontend/lib/api.js`
- `frontend/app/api/[...path]/route.js`
- Mongoose schema enums
- Global layout providers
- Billing webhook parsing
- Shared generated entity controllers
- RBAC policy handlers

Footer: NexusCRM Developer Guide | Chapter 16 | Validation, QA, And Smoke Testing

Chapter 17

Troubleshooting And Operational Playbooks

Chapter 17: Troubleshooting And Operational Playbooks

Login Timeout

Symptom:

```
Request timed out. The Render backend may be cold starting or API_URL/NEXT_PUBLIC_API_URL may be incorrect.
```

Check:

- Backend is running.
- `/api/health` works.
- Vercel `API_URL` points to Render backend root.
- Vercel `NEXT_PUBLIC_API_URL` points to Render backend root.
- Render `FRONTEND_URL` and `CORS_ORIGIN` match Vercel frontend.
- MongoDB Atlas is reachable from Render.

MongoDB SRV DNS Failure

Symptom:

```
querySrv ECONNREFUSED _mongodb._tcp...
```

Fix:

- Use valid Atlas URI.
- Confirm network access in Atlas.
- Use explicit DNS servers through `DNS_SERVERS`.
- Use `DNS_SERVERS=system` only if the host DNS is reliable.

Port Already In Use

Symptom:

```
listen EADDRINUSE :::4000
```

Fix:

- Stop the existing backend process.
- Use a different `PORT`.
- Confirm no stale Node process is listening.

Vercel API Misconfiguration

Symptom:

```
API server is not configured. Set API_URL or NEXT_PUBLIC_API_URL.
```

Fix:

- Set `API_URL`.
- Set `NEXT_PUBLIC_API_URL`.
- Redeploy Vercel.

Stripe Webhook Fails

Check:

- STRIPE_WEBHOOK_SECRET is correct.
- Webhook route is /api/billing/webhook.
- Raw body parser remains before normal JSON parser.
- Stripe CLI forwards to the correct local route.

Emails Not Sending

Check:

- Brevo key exists.
- Sender email is verified.
- Tenant SMTP/IMAP credentials are correct.
- Provider app passwords are used where required.

Socket Notifications Not Appearing

Check:

- NEXT_PUBLIC_WS_URL.
- Backend /socket.io reachable.
- JWT exists in localStorage.
- Browser console for socket auth errors.
- Notification records are being created in MongoDB.

Tenant Route Mismatch

Check:

- crm_tenant localStorage value.
- URL tenant segment.
- /api/auth/me tenant response.
- Tenant exists and is active.

Build Errors

Common causes:

- Incorrect relative imports.
- Client component hooks in server-only files.
- Missing environment assumptions during static generation.
- Next.js route handler API changes.

Fix:

- Run frontend lint.
- Run frontend build.

- Check route imports and client/server boundaries.

Footer: NexusCRM Developer Guide | Chapter 17 | Troubleshooting And Operational Playbooks

Chapter 18

Contribution Standards And Extension Patterns

Chapter 18: Contribution Standards And Extension Patterns

Engineering Standards

Developers should:

- Follow existing local patterns.
- Keep tenant isolation intact.
- Avoid broad refactors inside feature work.
- Add backend and frontend checks for permissions.
- Use shared components and API helpers.
- Add notifications/activity for important actions.
- Validate frontend lint/build before pushing.
- Smoke test backend startup after backend module changes.

Adding A Backend Module

Recommended steps:

- Create schema under `modules/{feature}/schemas`.
- Create service.
- Create controller.
- Create module.
- Register Mongoose models.
- Add guards/policies.
- Import module into `app.module.js`.
- Add activity/notifications if user-facing.
- Add frontend API helper.
- Add page/component.

Adding A Frontend Feature Page

Recommended steps:

- Place route under `frontend/app/[tenant]/...`
- Use `useSession()` when tenant/profile/rules are needed.
- Use `Can` for permission gates.
- Use feature API helper in `frontend/lib`.
- Use shared UI primitives.
- Add loading/error/empty states.
- Add route to `frontend/lib/navigation.js` if needed.

Adding A Generic Extension Entity

Use existing extension patterns:

- Define schema.
- Add entity config.
- Use generated controller/service when CRUD is enough.
- Add custom service/controller only when workflows are domain-specific.

Generic controller supports:

- CRUD.
- Bulk operations.
- Download.
- Run actions.
- RBAC subject mapping.

Adding Permissions

Steps:

- Add subject in `backend/src/common/constants/roles.js`.
- Update default group templates.
- Add policy handler if needed.
- Apply backend guards.
- Add frontend action and subject to `navigation/actions`.
- Test with restricted role.

Adding Notifications

Use `NotificationsService` when available. If the current module does not import realtime services, either import the notification schema and emit helper carefully or wire `RealtimeModule` where appropriate.

Notification payload should include:

- `type`
- `title`
- `body`
- `href`
- `entityType`
- `entityId`

Documentation Standards

When adding major functionality:

- Update relevant docs.
- Include route names.
- Include env variables.
- Include setup instructions.
- Include validation steps.
- Include troubleshooting notes when deployment behavior changes.

Appendix A: Quick Command Reference

Install

```
npm run install:all
```

Development

```
npm run dev:backend  
npm run dev:frontend
```

Frontend

```
npm run lint --prefix frontend  
npm run build --prefix frontend
```

Backend

```
npm run start --prefix backend  
npm run start:prod --prefix backend  
npm run flush-db --prefix backend
```

Health

```
node -e "fetch('http://127.0.0.1:4000/api/health').then(async r=>{console.log(r.status); console.log(await r.text())})"
```

Git

```
git status --short  
git diff --stat  
git log -5 --oneline
```

Footer: NexusCRM Developer Guide | Appendix A | Quick Command Reference

Appendix B: Developer Checklists

New Backend Feature Checklist

- ☐ Schema includes tenant scope.
- ☐ Service filters by tenant.
- ☐ Controller has guards/policies.
- ☐ Activity events added where useful.
- ☐ Notifications added for user-facing actions.
- ☐ Module imports Mongoose models correctly.
- ☐ `app.module.js` imports module when needed.
- ☐ Backend smoke starts cleanly.

New Frontend Feature Checklist

- ☐ Route is in correct app segment.
- ☐ Page uses API helper.
- ☐ Loading state exists.
- ☐ Error state exists.
- ☐ Empty state exists.
- ☐ RBAC gate exists when needed.
- ☐ Navigation entry has action/subject when needed.
- ☐ Frontend lint passes.
- ☐ Frontend build passes.

Deployment Checklist

- ☐ Backend Render env set.
- ☐ Frontend Vercel env set.
- ☐ MongoDB Atlas network access configured.
- ☐ `JWT_SECRET` is strong.
- ☐ Superadmin credentials changed.
- ☐ `CORS_ORIGIN` matches Vercel URL.
- ☐ `API_URL` points to Render backend root.
- ☐ `/api/health` returns OK.
- ☐ Signup/login works.
- ☐ Socket notifications work.

Release Checklist

- ☐ Review git diff.
- ☐ Run frontend lint.

- ☐ Run frontend build.
- ☐ Run backend smoke.
- ☐ Confirm no secrets are staged.
- ☐ Commit with accurate message.
- ☐ Push to GitHub.
- ☐ Verify deployment pipeline.

Footer: NexusCRM Developer Guide | Appendix B | Developer Checklists

End Of Document

NexusCRM Developer Guide

Built for engineers maintaining a production-grade multi-tenant SaaS CRM.